

Excelente pergunta — este é exatamente o tipo de base conceitual que todo estudante de **Ciência da Computação** precisa dominar antes de avançar em **Programação Orientada a Objetos em Java**.

Vou estruturar a explicação como faríamos em uma **aula universitária**, dividindo em seções bem organizadas.

1. Introdução à Plataforma Java

1.1 Origem e História da Linguagem Java

A linguagem **Java** foi criada no início da década de **1990** pela empresa **Sun Microsystems**.

Criadores principais

O projeto foi liderado por:

- **James Gosling**
- **Mike Sheridan**
- **Patrick Naughton**

Eles faziam parte de um grupo interno chamado:

Green Team

Contexto tecnológico da época

No início dos anos **90**, havia um grande problema na indústria de software:

- programas eram **fortemente dependentes do sistema operacional**
- softwares feitos para **Windows não funcionavam em Unix ou Mac**
- cada arquitetura exigia **recompilação ou reescrita**

O objetivo do projeto era criar uma linguagem que fosse:

- **portável**
- **segura**
- **independente de plataforma**

Inicialmente, Java **não foi criado para a internet**, mas sim para **dispositivos eletrônicos embarcados**, como:

- televisões interativas
- sistemas domésticos inteligentes

O nome original da linguagem era:

Oak

Posteriormente foi renomeada para **Java**.

1.2 O Grande Marco: A Internet

Em **1995**, a Sun lançou oficialmente o Java com o slogan:

"Write Once, Run Anywhere"

Isso foi revolucionário porque os programas Java eram compilados para:

Bytecode

Esse bytecode roda em uma máquina virtual chamada:

JVM (Java Virtual Machine)

Isso permitiu executar o mesmo programa em:

- Windows
- Linux
- Mac
- servidores
- dispositivos embarcados

sem recompilar.

2. Linha do Tempo da Evolução do Java

Ano	Versão	Principais mudanças
-----	--------	---------------------

1995	Java 1.0	Primeira versão oficial
1998	Java 2 (1.2)	Introdução do Swing e Collections
2004	Java 5	Generics, Annotations, Enum
2011	Java 7	Try-with-resources
2014	Java 8	Lambda e Streams (revolução funcional)
2017	Java 9	Sistema de módulos
2018	Java 11	LTS, melhorias em HTTP Client
2021	Java 17	LTS moderna
2023	Java 21	LTS atual com melhorias de performance

3. Java vs Outras Linguagens da Mesma Época

Language	Característica
C	Muito rápida, mas complexa
C++	Orientada a objetos, mas difícil de manter
Perl	Muito usada para scripts web
Java	Segurança + Portabilidade + POO

Por que Java venceu?

Principais fatores:

- ❶ Portabilidade (JVM)
- ❷ Orientação a objetos forte
- ❸ Grande biblioteca padrão

- 4 Segurança para internet
 - 5 Comunidade enorme
-

4. Versões LTS vs Não-LTS

LTS (Long Term Support)

São versões com **suporte prolongado**.

Exemplos:

Java 8
Java 11
Java 17
Java 21

Empresas utilizam LTS porque:

- recebem atualizações por **anos**
 - são mais **estáveis**
-

Versões não-LTS

Exemplo:

Java 12
Java 13
Java 14
Java 15

Servem para:

- testar novas funcionalidades
 - evoluir a linguagem rapidamente
-

5. Impacto do Java nas Empresas

Java se tornou dominante em:

Sistemas corporativos

- bancos
- sistemas governamentais
- telecomunicações
- e-commerce

Tecnologias baseadas em Java:

- **Spring Framework**
- **Hibernate**
- **Android**
- **Kafka**
- **Elasticsearch**

Grandes empresas que usam Java:

- Amazon
 - Netflix
 - LinkedIn
 - Uber
 - bancos no mundo inteiro
-

6. Por que Java continua relevante?

Java continua relevante porque possui:

1 Estabilidade

É usado há **mais de 25 anos**.

2 Ecossistema gigantesco

Frameworks:

- Spring Boot
 - Jakarta EE
 - Quarkus
 - Micronaut
-

3 Performance

Com otimizações da **JVM** e **JIT Compiler**.

4 Comunidade global

Milhões de desenvolvedores.

5 Presença em múltiplos ambientes

- servidores
 - cloud
 - Android
 - big data
-

7. Características da Linguagem Java

Principais características:

Característica	Explicação
Orientada a objetos	Organização por classes
Portável	JVM executa bytecode
Segura	gerenciamento automático de memória
Robusta	forte verificação de erros

8. Pilares da Programação Orientada a Objetos

Os 4 pilares da POO:

- 1 Encapsulamento
- 2 Herança
- 3 Polimorfismo
- 4 Abstração

Vamos ver um exemplo simples.

9. Exemplo de Código Demonstrando POO

```
// Classe abstrata representando um Animal
abstract class Animal {
```

```
    // Encapsulamento: atributo protegido
    protected String nome;
```

```
    // Construtor
    public Animal(String nome) {
        this.nome = nome;
    }
```

```
    // Método abstrato (Abstração)
    public abstract void emitirSom();
}
```

```
// Classe que herda de Animal
class Cachorro extends Animal {
```

```
    public Cachorro(String nome) {
        super(nome);
    }
```

```

// Polimorfismo (sobrescrita de método)
@Override
public void emitirSom() {
    System.out.println(nome + " diz: Au Au!");
}
}

// Outra classe que herda de Animal
class Gato extends Animal {

    public Gato(String nome) {
        super(nome);
    }

    @Override
    public void emitirSom() {
        System.out.println(nome + " diz: Miau!");
    }
}

// Classe principal
public class Main {

    public static void main(String[] args) {

        Animal a1 = new Cachorro("Rex");
        Animal a2 = new Gato("Mimi");

        // Polimorfismo
        a1.emitirSom();
        a2.emitirSom();
    }
}

```

Explicação do Código

Encapsulamento

protected String nome;

Os dados são controlados dentro da classe.

Herança

`class Cachorro extends Animal`

Cachorro herda propriedades de Animal.

Abstração

`abstract class Animal`

Define um comportamento geral.

Polimorfismo

`a1.emitirSom();`

Cada objeto executa seu próprio comportamento.

10. Ferramentas da Plataforma Java

JVM

Executa o bytecode.

JRE

Ambiente necessário para rodar programas Java.

Contém:

- JVM
 - bibliotecas padrão
-

JDK

Kit de desenvolvimento Java.

Contém:

- compilador
 - ferramentas
 - JRE
-

11. Principais Ferramentas do JDK

Ferramenta	Função
javac	compila código
java	executa programa
jar	cria pacotes
javap	inspeciona bytecode
jshell	terminal interativo

Exemplo de Compilação

Arquivo:

Hello.java

Compilar:

javac Hello.java

Executar:

java Hello

12. Estrutura Básica de um Programa Java

Exemplo:

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
  
        System.out.println("Olá, Mundo!");  
  
    }  
}
```

Componentes

public class

Define a classe.

main()

Ponto de entrada do programa.

System.out.println

Imprime no console.

13. Java e os Níveis de Linguagem

Linguagem de máquina

Executada diretamente pela CPU.

Exemplo:

```
10110000 01100001
```

Assembly

Representação simbólica da linguagem de máquina.

Exemplo:

```
MOV AX, 1  
INT 21H
```

Linguagem de alto nível

Mais próxima do pensamento humano.

Exemplo:

```
System.out.println("Hello");
```

Exemplo Comparativo Hello World

Assembly (simplificado)

```
section .data  
msg db 'Hello World',0
```

```
section .text  
global _start
```

```
_start:
```

```
mov edx,11
mov ecx,msg
mov ebx,1
mov eax,4
int 0x80
```

Java

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World");
    }
}
```

14. Bytecode e JVM

Fluxo de execução:

```
Código Java (.java)
↓
Compilador (javac)
↓
Bytecode (.class)
↓
JVM
↓
Sistema Operacional
```

15. Por que um Cientista da Computação Deve Entender Java?

Porque Java ensina conceitos fundamentais:

- POO

- arquitetura de software
- sistemas distribuídos
- concorrência
- engenharia de software

Compreender Java ajuda a entender:

- como linguagens funcionam
 - como sistemas grandes são construídos
 - como frameworks funcionam
-

Conclusão

Java se tornou uma das linguagens mais importantes da história da computação devido a:

- portabilidade
- segurança
- robustez
- forte suporte a **POO**
- enorme ecossistema corporativo

A evolução constante da linguagem e da **JVM** mantém Java relevante até hoje.

Referências Acadêmicas

GOSLING, James; JOY, Bill; STEELE, Guy.
The Java Language Specification. Oracle, 2023.

BLOCH, Joshua.
Effective Java. Addison-Wesley, 2018.

SIERRA, Kathy; BATES, Bert.
Head First Java. O'Reilly, 2005.

ECKEL, Bruce.
Thinking in Java. Prentice Hall, 2006.

Verificação de Compreensão

Para continuar seu aprendizado, gostaria de saber:

1) Você já programou em **Java antes ou está começando agora?**

2) Você gostaria que eu explicasse também:

- **Como funciona a JVM internamente**
- **Memória do Java (Stack vs Heap)**
- **Arquitetura completa da plataforma Java**
- **Exercícios práticos de POO em Java**

Se quiser, também posso mostrar algo muito interessante:

Como um código Java vira bytecode dentro da JVM (passo a passo). 🚀